

# Azure AD Authentication with Angular SPA and .NET Web API

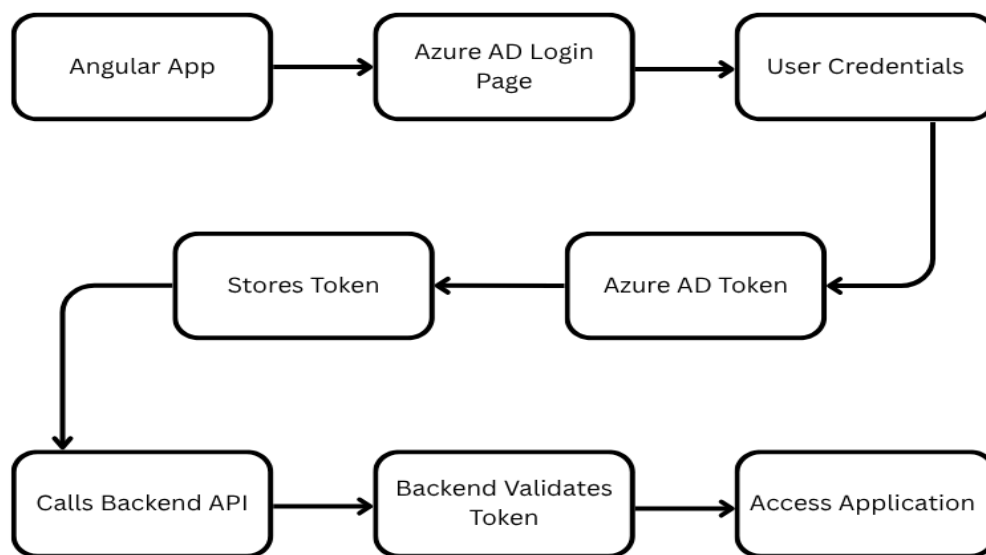
## 1. Overview

- Azure Active Directory (Azure AD) is Microsoft's cloud-based identity and access management service.
- It enables users to authenticate using their Microsoft accounts (work or school), providing **secure login** and **single sign-on (SSO)** for applications.

This document explains how to integrate Azure AD login into a web application (Angular frontend + .NET backend), the implementation steps, advantages, and limitations.

---

## 2. Flow Diagram



## 3. Steps to Implement Azure AD Login

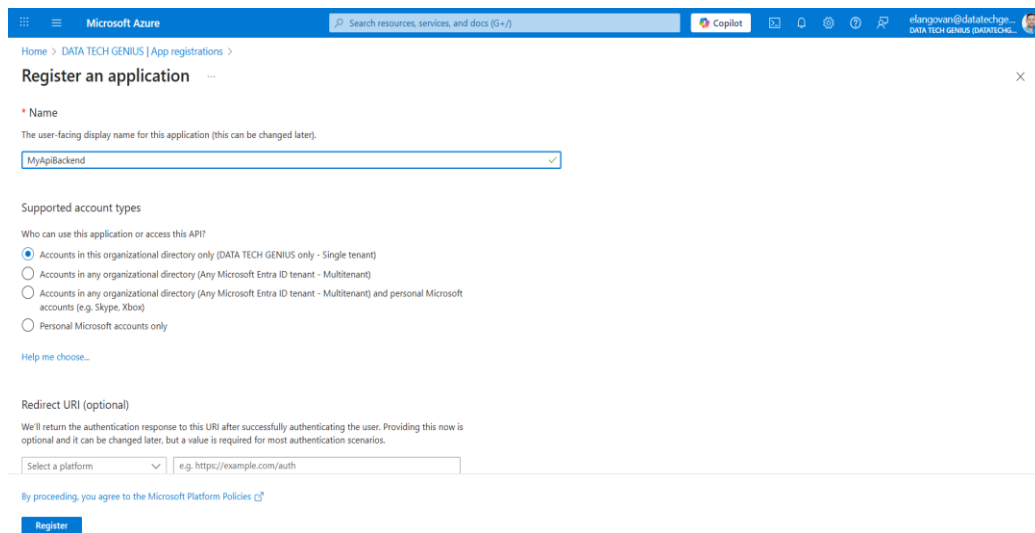
### Step-1: Azure AD App Registrations

You need **two app registrations**:

1. **API app (Backend)** – exposes scopes for authorization.
2. **SPA(Single Page Application) app (Frontend)** – requests access tokens for the API.

#### 1. API App (Backend)

- Navigate to **Azure Portal** → **Azure Active Directory** → **App registrations** → **New registration**.
- Fill in the required details:
  - **Name:** Application name (MyApiBackend)
  - **Supported account types:** Single-tenant or Multi-tenant
  - **Redirect URI:** None required
  - **After creation:** Note **Application (client) ID** → **API\_CLIENT\_ID**



The screenshot shows the 'Register an application' page in the Azure Portal. The page has a blue header with the Microsoft Azure logo and a search bar. Below the header, the breadcrumb navigation is 'Home > DATA TECH GENIUS | App registrations >'. The main heading is 'Register an application'. The form contains the following sections:

- Name:** A text input field with the value 'MyApiBackend'.
- Supported account types:** A section titled 'Who can use this application or access this API?' with four radio button options:
  - ☒ Accounts in this organizational directory only (DATA TECH GENIUS only - Single tenant)
  - ☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant)
  - ☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)
  - ☐ Personal Microsoft accounts only
- Redirect URI (optional):** A section titled 'We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.' It includes a dropdown menu for 'Select a platform' and a text input field with the value 'e.g. https://example.com/auth'.

At the bottom of the form, there is a link 'By proceeding, you agree to the Microsoft Platform Policies' and a blue 'Register' button.

- Click **Register**

Microsoft Azure

Home > DATA TECH GENIUS | App registrations >

# MyApiBackend

Search

Delete Endpoints Preview features

Overview

- Quickstart
- Integration assistant
- Diagnose and solve problems
- Manage
  - Branding & properties
  - Authentication
  - Certificates & secrets
  - Token configuration
  - API permissions
  - Expose an API
  - App roles
  - Owners
  - Roles and administrators
  - Manifest
- Support > Troubleshooting

Got a second? We would love your feedback on Microsoft identity platform (previously Azure AD for developer). →

## Essentials

|                         |                                        |                             |                                                              |
|-------------------------|----------------------------------------|-----------------------------|--------------------------------------------------------------|
| Display name            | : MyApiBackend                         | Client credentials          | : <a href="#">Add a certificate or secret</a>                |
| Application (client) ID | : 32fa6f18-303b-45a0-9e3c-d80bef0f5c91 | Redirect URIs               | : <a href="#">Add a Redirect URI</a>                         |
| Object ID               | : 40242855-72a3-49f4-844d-ba49a9fdd1f6 | Application ID URI          | : <a href="#">api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91</a> |
| Directory (tenant) ID   | : f1a8e259-12b0-4aa9-a8e5-2ce5c6b98233 | Managed application in L... | : <a href="#">MyApiBackend</a>                               |
| Supported account types | : <a href="#">My organization only</a> |                             |                                                              |

Starting June 30th, 2020 we will no longer add any new features to Azure Active Directory Authentication Library (ADAL) and Azure Active Directory Graph. We will continue to provide technical support and security updates but we will no longer provide feature updates. Applications will need to be upgraded to Microsoft Authentication Library (MSAL) and Microsoft Graph. [Learn more](#)

[Get Started](#) [Documentation](#)

### Build your application with the Microsoft identity platform

The Microsoft identity platform is an authentication service, open-source libraries, and application management tools. You can create modern, standards-based authentication solutions, access and protect APIs, and add sign-in for your users and customers. [Learn more](#)

## Expose an API:

- Go to **Expose an API**
- Set **Application ID URI**: `api://<API_CLIENT_ID>`
- Add a scope:
  - **Scope name**: `access_as_user`
  - **Who can consent**: Admins and users
  - **Display name**: Access MyApi
  - **Description**: Allow app to access MyApi as signed-in user

Microsoft Azure

Home > DATA TECH GENIUS > App registrations > MyApiBackend

### MyApiBackend | Expose an API

Search resources, services, and docs (G+)

Overview

Quickstart

Integration assistant

Diagnose and solve problems

Manage

- Branding & properties
- Authentication
- Certificates & secrets
- Token configuration
- API permissions
- Expose an API**
- App roles
- Owners
- Roles and administrators
- Manifest

Support + Troubleshooting

Add or remove favorites by pressing Ctrl+shift+F

Application ID URI: `api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91` Edit

#### Scopes defined by this API

Define custom scopes to restrict access to data and functionality protected by the API. An application that requires access to this API can request that a user or admin consent to one or more of these.

Adding a scope here creates only delegated permissions. If you are looking to create application-only scopes, use 'App roles' and define app roles assignable to application type. [Go to App roles](#).

+ Add a scope

| Scopes                                                             | Who can consent  | Admin consent display name |
|--------------------------------------------------------------------|------------------|----------------------------|
| <code>api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91/access_...</code> | Admins and users | Read user Files            |
| <code>api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91/access_...</code> | Admins only      | Access MyApi               |

#### Authorized client applications

Authorizing a client application indicates that this API trusts the application and users should not be asked to consent to this API.

+ Add a client application

| Client ID | Scopes |
|-----------|--------|
|-----------|--------|

#### Add a scope

Scope name \*

`Access-API`

`api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91/Access-API`

Who can consent? Admins and users Admins only

Admin consent display name \*

`Read Files`

Admin consent description \*

`Allow signed user to read`

User consent display name

`e.g. Read your files`

User consent description

`e.g. Allows the app to read your files.`

State

Enabled Disabled

Add scope Cancel

- Full scope identifier:  
`api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91/access_as_user`

Microsoft Azure

Home > DATA TECH GENIUS > App registrations > MyApiBackend

### MyApiBackend | Expose an API

Search resources, services, and docs (G+)

Overview

Quickstart

Integration assistant

Diagnose and solve problems

Manage

- Branding & properties
- Authentication
- Certificates & secrets
- Token configuration
- API permissions
- Expose an API**
- App roles
- Owners
- Roles and administrators
- Manifest

Application ID URI: `api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91` Edit

#### Scopes defined by this API

Define custom scopes to restrict access to data and functionality protected by the API. An application that requires access to parts of this API can request that a user or admin consent to one or more of these.

Adding a scope here creates only delegated permissions. If you are looking to create application-only scopes, use 'App roles' and define app roles assignable to application type. [Go to App roles](#).

+ Add a scope

| Scopes                                                             | Who can consent  | Admin consent display name | User consent display name | State   |
|--------------------------------------------------------------------|------------------|----------------------------|---------------------------|---------|
| <code>api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91/access_...</code> | Admins and users | Read user Files            |                           | Enabled |
| <code>api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91/access_...</code> | Admins only      | Access MyApi               |                           | Enabled |

#### Authorized client applications

Authorizing a client application indicates that this API trusts the application and users should not be asked to consent when the client calls this API.

## 2. SPA App (Frontend)

- Navigate to **Azure Portal** → **Azure Active Directory** → **App registrations** → **New registration**.
- Fill in the required details:
  - **Name:** Application name (AzureLoginApp)
  - **Supported account types:** Single-tenant or Multi-tenant
  - **Redirect URI:** Application URL to receive login response (<http://localhost:4200>)
  - Note **Application (client) ID** → **SPA\_CLIENT\_ID**

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

Home > DATA TECH GENIUS | App registrations >

### Register an application

AzureLoginApp ✓

Supported account types

Who can use this application or access this API?

☒ Accounts in this organizational directory only (DATA TECH GENIUS only - Single tenant)

☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant)

☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)

☐ Personal Microsoft accounts only

[Help me choose...](#)

Redirect URI (optional)

We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.

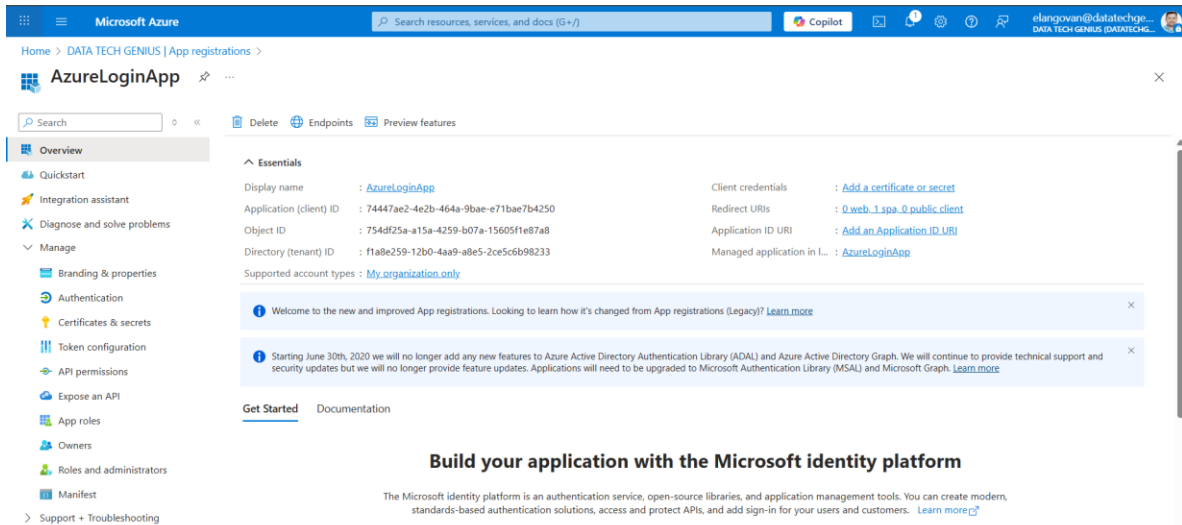
Single-page application (SPA) ✓

Register an app you're working on here. Integrate gallery apps and other apps from outside your organization by adding from [Enterprise applications](#).

By proceeding, you agree to the [Microsoft Platform Policies](#)

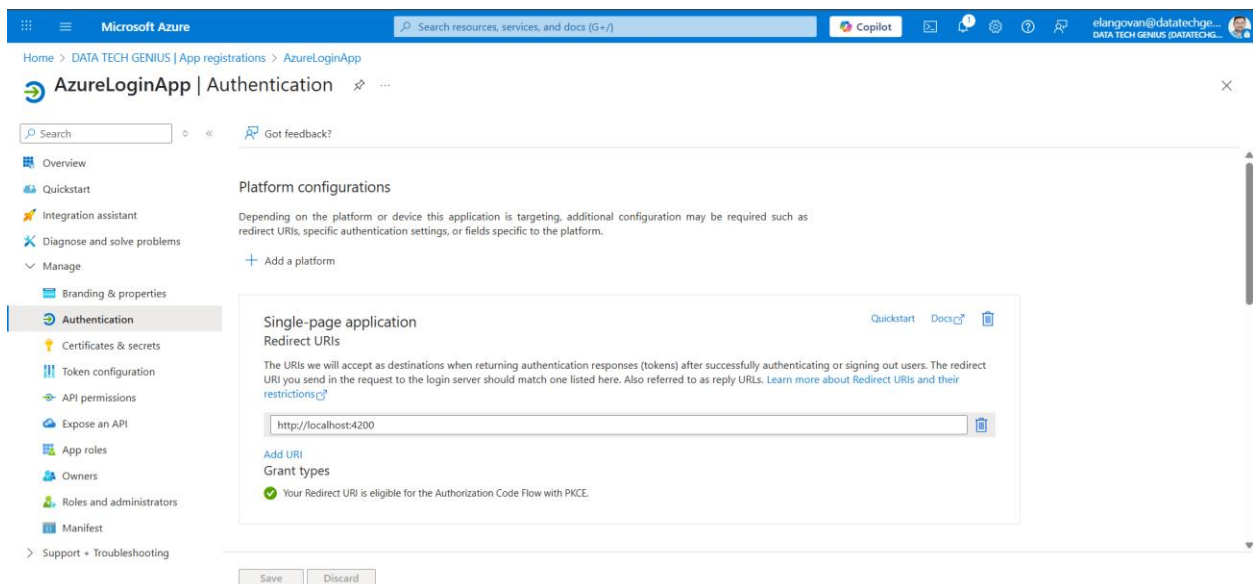
**Register**

- Click **Register**.



## Configure Authentication

1. Go to **Authentication** → **Add a platform** → **Single-page application (SPA)**.
2. Enter redirect URI.
3. Enable **ID tokens** and **Access tokens** (if the application calls APIs).
4. Development: `http://localhost:4200`  
Production: `https://azureLoginApp.com`



## Add API Permissions

1. Navigate to **API Permissions** → **Add a permission**.
2. Choose **MyApi** → select **delegated scope**
3. Select the Permission **access\_as\_user**
4. Click **Grant admin consent** to approve permissions.

The screenshot shows the 'Request API permissions' dialog in the Azure portal. The dialog is titled 'Request API permissions' and contains the following sections:

- Your application needs to access the APIs as the signed-in user.**
- Application permissions:** Your application runs as a background service or daemon without a signed-in user.
- Select permissions:** A search bar and a list of permissions.
- Permissions (1):** A table showing the selected permission.

| Permission                                                         | Admin consent required |
|--------------------------------------------------------------------|------------------------|
| <input type="checkbox"/> access-api-scope<br>Read user files       | No                     |
| <input checked="" type="checkbox"/> access_as_user<br>Access MyApi | Yes                    |

Buttons at the bottom: **Add permissions** and **Discard**.

The screenshot shows the 'API permissions' page in the Azure portal. The page displays the configured permissions for the application 'AzureLoginApp'.

**Configured permissions**

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

**API / Permissions name** | **Type** | **Description** | **Admin consent required** | **Status**

|                     |           |                               |     |                           |
|---------------------|-----------|-------------------------------|-----|---------------------------|
| Microsoft Graph (1) |           |                               |     |                           |
| User.Read           | Delegated | Sign in and read user profile | No  | ...                       |
| MyApiBackend (1)    |           |                               |     |                           |
| access_as_user      | Delegated | Access MyApi                  | Yes | Not granted for DATA T... |

To view and manage consented permissions for individual apps, as well as your tenant's consent settings, try [Enterprise applications](#).

## Step-2: Backend (.NET Web API)

### Create API

```
dotnet new webapi -n MyApi
```

```
cd MyApi
```

### Install NuGet packages:

```
dotnet add package Microsoft.Identity.Web
```

```
dotnet add package Microsoft.Identity.Web.Resource
```

### Program.cs (minimal hosting)

```
// Authentication & Authorization

builder.Services.AddAuthentication("Bearer")

.AddMicrosoftIdentityWebApi(builder.Configuration.GetSection("AzureAd"));

builder.Services.AddAuthorization();

// Enable CORS for Angular

builder.Services.AddCors(options =>

{

    options.AddPolicy("AllowAll", policy =>

    {

        policy.AllowAnyOrigin()

            .AllowAnyHeader()

            .AllowAnyMethod();

    });

});

app.UseCors();

app.UseAuthentication();

app.UseAuthorization();
```

## appsettings.json

```
"AzureAd": {  
  "Instance": "https://login.microsoftonline.com/",  
  "TenantId": "f1a8e259-12b0-4aa9-a8e5-2ce5c6b98233",  
  "ClientId": "32fa6f18-303b-45a0-9e3c-d80bef0f5c91",  
  "Audience": "api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91"  
}
```

## WeatherController.cs

```
[HttpGet("public")]  
public IActionResult Public() => Ok(new { msg = "This is public" });  
  
[HttpGet("private")]  
[Authorize]  
public IActionResult Private()  
{  
    var user = User?.Identity?.Name ?? "unknown";  
    return Ok(new { msg = $"Hello {user}, this is protected" });  
}
```

## Start API:

```
dotnet run
```

- Now listening on: <http://localhost:5141>
-

## Step-3: Frontend (Angular SPA with MSAL)

### Create Angular app

```
npm install -g @angular/cli  
  
ng new my-angular-spa --routing --style=scss  
  
cd my-angular-spa
```

### Install packages:

```
npm install @azure/msal-browser @azure/msal-angular
```

### MSAL Configuration (msal-config.ts)

```
export const msalConfig = {  
  auth: {  
    clientId: '74447ae2-4e2b-464a-9bae-e71bae7b4250',  
    authority: 'https://login.microsoftonline.com/f1a8e259-12b0-4aa9-a8e5-2ce5c6b98233',  
    redirectUri: 'http://localhost:4200',  
  },  
  cache: {  
    cacheLocation: BrowserCacheLocation.LocalStorage,  
    storeAuthStateInCookie: false,  
  },  
};  
  
export const protectedResources = {  
  api: {  
    endpoint: 'http://localhost:5141/api',  
    scopes: ['api://32fa6f18-303b-45a0-9e3c-d80bef0f5c91/access-api-scope'],  
  },  
};
```

```
};

export async function MSALInstanceFactory() {

  return new (await import('@azure/msal-
browser')).PublicClientApplication(msalConfig);

}
```

## App Main (main.ts)

```
export function MSALInstanceFactory() {

  return new PublicClientApplication(msalConfig);

}

export function MSALInterceptorConfigFactory() {

  return {

    interactionType: InteractionType.Redirect,

    protectedResourceMap: new Map([

      [protectedResources.api.endpoint, protectedResources.api.scopes]

    ])

  };

}

async function bootstrap() {

  const msalInstance = MSALInstanceFactory();

  await msalInstance.initialize();

  await bootstrapApplication(AppComponent, {

    providers: [

      importProvidersFrom(HttpClientModule),

      { provide: MSAL_INSTANCE, useValue: msalInstance },

      { provide: MSAL_INTERCEPTOR_CONFIG, useFactory:
MSALInterceptorConfigFactory },

      { provide: HTTP_INTERCEPTORS, useClass: MsalInterceptor, multi:
true },

    ]

  });

}
```

```

        MsalService, MsalGuard, MsalBroadcastService
    ],
    });
}

bootstrap().catch(err => console.error(err));

```

## App Component (app.component.ts)

```

export class AppComponent implements OnInit {
    loggedIn = false;
    username = '';
    result: any;
    private interactionInProgress = false;
    constructor(
        private msalService: MsalService,
        private msalBroadcast: MsalBroadcastService,
        private http: HttpClient
    ) {}
    ngOnInit() {
        this.msalBroadcast.inProgress$
            .pipe(filter(status => status === InteractionStatus.Startup ||
status === InteractionStatus.Login))
            .subscribe(() => this.interactionInProgress = false);
        this.msalService.instance.handleRedirectPromise()
            .then(result => {
                if (result?.account) {
                    this.loggedIn = true;
                    this.username = result.account.username;
                }
            })
    }
}

```

```

    })

    .catch(err => console.error('Redirect error', err));

const accounts = this.msalService.instance.getAllAccounts();
if (accounts.length > 0) {
    this.loggedIn = true;
    this.username = accounts[0].username;
}
}

async login() {
    if (this.interactionInProgress) {
        console.warn('Login skipped: another interaction is in progress');
        return;
    }

    await this.msalService.instance.initialize();
    this.msalService.loginRedirect();
}

logout() {
    this.msalService.logoutRedirect();
}

callApi() {
    console.log("Calling API...");

    this.http.get(`${protectedResources.api.endpoint}/weather/private`).subscribe(
        res => {
            console.log('API response:', res);
            this.result = res;
        }
    );
}

```

```

    },
    err => {
        console.error('API error:', err);
        this.result = err;
    }
}
}

```

## App Template (app.component.html)

```

<div style="padding:20px">
    <h2>Angular + Azure AD Login + API</h2>

    <div *ngIf="!loggedIn">
        <button (click)="login()">Login</button>
    </div>

    <div *ngIf="loggedIn">
        <p>Welcome: {{ username }}</p>
        <button (click)="logout()">Logout</button>
    </div>

    <hr />

    <button (click)="callApi()">Call Protected API</button>
    <pre>{{ result | json }}</pre>
</div>

```

## Run Angular:

ng serve

Running on Port: <http://localhost:4200>

- **Login:** Redirect to Azure AD
  - **Call Protected API:** MSAL attaches Authorization: Bearer <access\_token> automatically
-

## Step-4: SPA → API Permission Flow

1. SPA requests **API delegated scope** (access\_as\_user)
2. Azure AD prompts for consent
3. Access token issued with audience (aud) = API App ID URI
4. Backend validates token via `Microsoft.Identity.Web`
5. Protected endpoints respond with user info

---

## Step-5: Testing the Flow

- Start backend: `dotnet run` → `http://localhost:5141`
  - Start frontend: `ng serve` → `http://localhost:4200`
  - Open SPA, login, and call **public** and **private** endpoints
  - MSAL interceptor automatically attaches the token
  - .NET API validates token and returns the data
-

## 4. Advantages

- **Single Sign-On (SSO):** One login for multiple apps.
  - **Security:** Supports OAuth 2.0, OpenID Connect, and MFA.
  - **Enterprise Integration:** Works with organizational accounts.
  - **No Password Storage:** Credentials handled by Azure AD.
  - **API Access:** Easy integration with Microsoft Graph and custom APIs.
- 

## 5. Disadvantages

- **Azure Dependency:** Requires Azure subscription and cloud accounts.
- **Setup Complexity:** Initial configuration can be tricky for beginners.
- **Internet Required:** Works only with online Azure AD accounts.
- **Learning Curve:** Requires understanding of OAuth 2.0/OpenID Connect.
- **Microsoft Ecosystem:** Mainly limited to Microsoft accounts.